# App Dev Project Report

## 1. Student Details

- **Name**: Debankush Ghosh
- **Roll Number**: 24f3001469
- **Email**: 24f3001469@ds.study.iitm.ac.in
- **About Me**: I am currently pursuing a BS in Data Science at IIT Madras at the diploma level. I have a strong interest in Python programming, database management, and building data-driven web applications using the Flask framework.

---

## 2. Project Details

- **Project Title**: CareerConnect - Placement Portal
- **Problem Statement**: To design and build a unified placement management system that streamlines recruitment by allowing admins to manage users, companies to post job drives, and students to apply with verified resumes.
- **Approach**: The application uses a role-based architecture (Admin, Company, Student). It features a secure resume upload system and utilizes SQL joins to provide recruiters with integrated views of student profiles and their uploaded documents.

---

## 3. AI/LLM Declaration

- I used **Gemini** to assist in debugging sqlite3.OperationalError schema mismatches, generating responsive CSS for dashboards, and structuring Flask routes for file uploads.
- The extent of AI/LLM usage is approximately **15-20%**, focused on UI/UX styling and troubleshooting database migration logic.
- All core logic, database schema design, and final integration were performed manually.

---

# 4. Technologies and Frameworks Used

| Technology / Library | Purpose |
|---|---|
| **Flask** | Core backend web framework |
| **SQLite** | Lightweight local database for storing user, drive, and application data |
| **Jinja2** | Template engine for rendering dynamic dashboards and tables |
| **SQLAlchemy** | ORM for database interactions and schema management |
| **Werkzeug** | Secure password hashing and filename sanitization |
| **PowerShell** | Local development environment and server management |

---

# 5. Database Schema / ER Diagram

**Tables:**

- **Students**: Stores profile details (id, name, email, password).
- **Companies**: Stores recruiter details and approval status (id, name, email, password, approved).
- **Drives**: Logs recruitment opportunities (id, company_id, title, description, deadline).
- **Applications**: Links students to drives with status and resume paths (id, student_id, drive_id, status, resume).

**Relationships:**

- **One-to-Many**: Company → Drives
- **One-to-Many**: Student → Applications
- **One-to-Many**: Drives → Applications

---

# 6. API Resource Endpoints

| Endpoint | Method | Description |
|---|---|---|
| **/student/register** | POST | Create a new student profile |
| **/company/add_drive** | POST | Submit a new recruitment drive for admin approval |
| **/student/apply/<int:drive_id>** | POST | Upload resume and apply for a specific drive |
| **/admin/approve_company** | GET | Update company status to active |

---

# 7. Architecture and Features

**Architecture Overview:**

- **app.py**: Main entry point containing route logic and database initialization.
- **/templates**: HTML files for Admin, Company, and Student dashboards.
- **/static/resumes**: Secure storage for student-uploaded PDF/DOC files.

**Implemented Features:**

- **Multi-User Authentication**: Separate login portals for Admins, Companies, and Students.
- **Resume Management**: Automated file naming and secure storage system.
- **Recruitment Workflow**: Drive creation, admin verification, and student application tracking.
- **Admin Dashboard**: Real-time statistics on registered students and active drives.